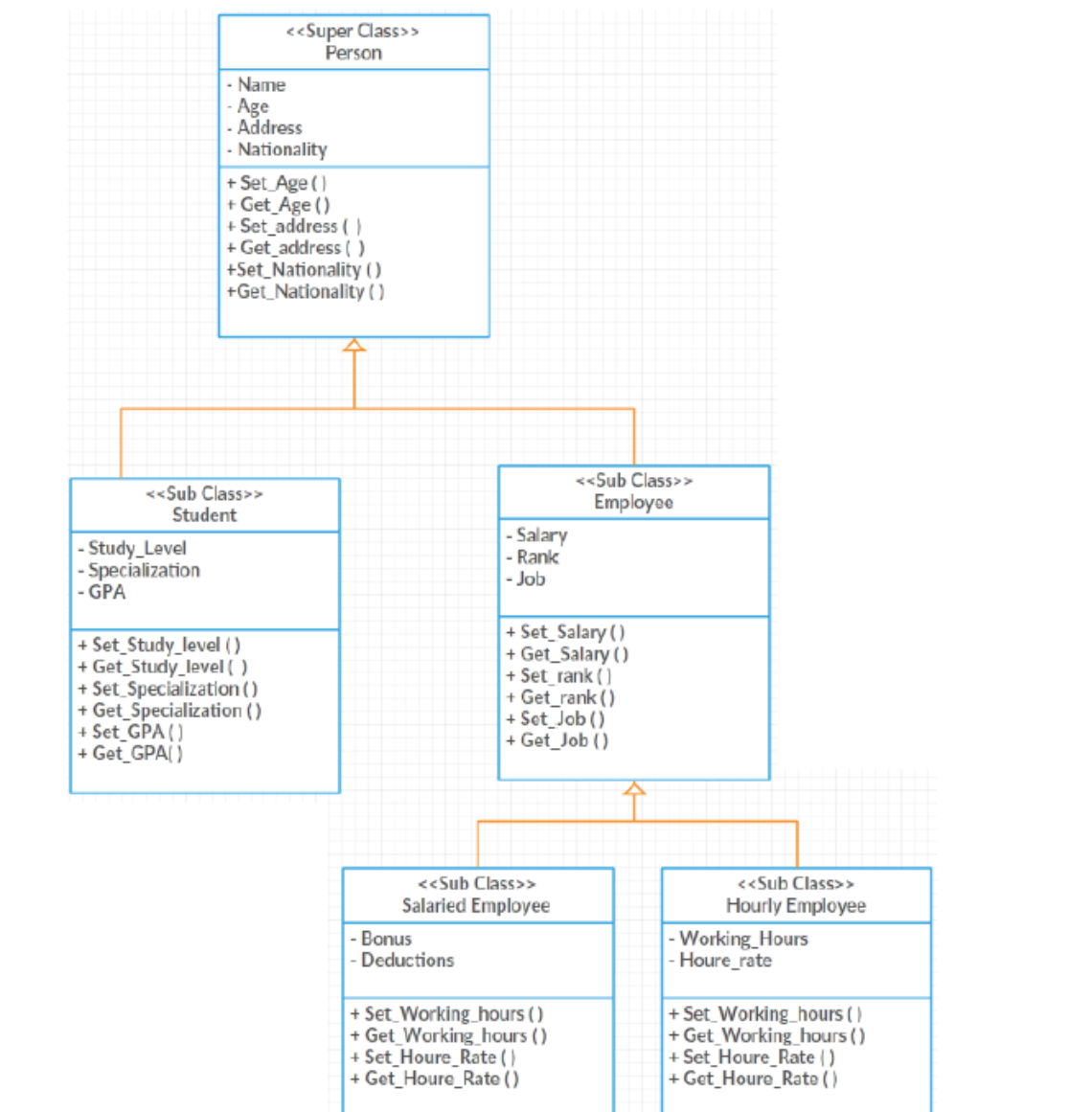
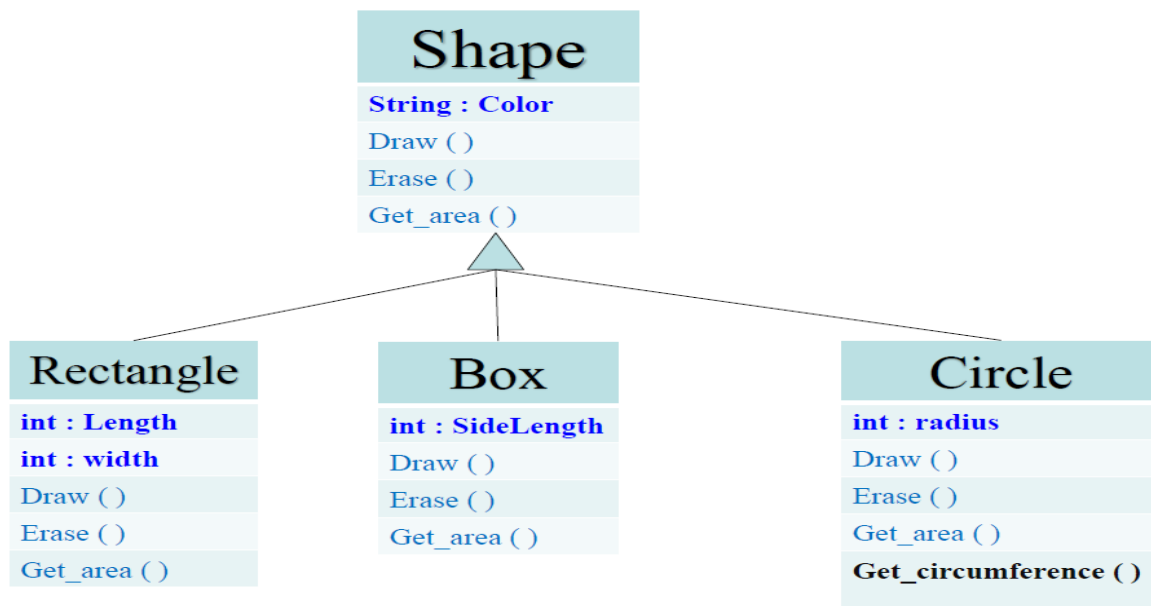


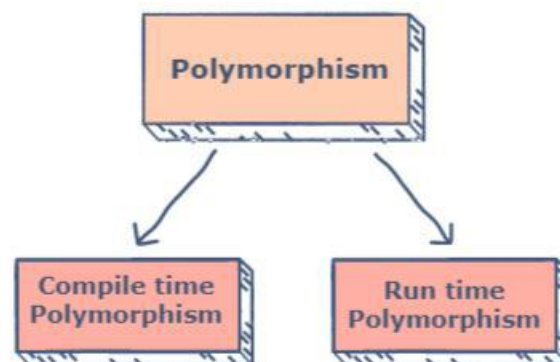
6-1 What is Polymorphism?

- ✚ **Polymorphism** is an object-oriented programming concept that refers to the ability of a *variable, function or object to take on multiple forms.*
- ✚ With **polymorphism**, class objects belonging to the same hierarchical tree (inherited from a common **parent class**) may have functions with the same name, but with different behaviors.





6-2 Types of Polymorphism



1- polymorphism at compile

Also known as **early binding** and **static polymorphism**, in compile-time polymorphism, the compiler determines how the function or operator will work depending on the context. This type of polymorphism is achieved by function overloading or operator overloading.

✚ **Example of polymorphism at compile time is Method overloading (function overloading):**

✚ Function overloading is a feature of object-oriented programming where two or more functions can have the same name but behave differently for different parameters. Such functions are said to be **overloaded**; hence, this is known as **Function Overloading**. Functions can be overloaded either by **changing the number of arguments** or **changing the type of arguments**.

Example :

```
#include <iostream>
using namespace std;

class compiletime {
public:

    // Function to add two integers
    void add(int a, int b) {
        cout << "Integer Sum = " << a + b
        << endl;
    }

    // Function to add two floating point values
    void add(double a, double b) {
        cout << "Float Sum = " << a + b
        << endl ;
    }
};

int main() {
    compiletime obj;

    // add() called with int values
    obj.add(10, 2);

    // add() called with double value
    obj.add(5.3, 6.2);

    return 0;
}
```

Integer Sum = 12

Float Sum = 11.5

H.W: What is the change between function overloading and operator overloading such (+ operator)? explain with C++ code.

2. Runtime Polymorphism

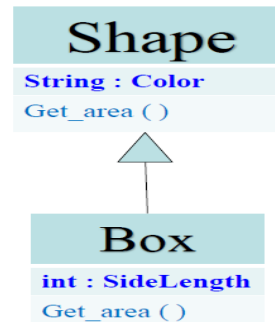
Also known as **late binding** and **dynamic polymorphism**, the function call in runtime polymorphism is resolved at runtime in contrast with compile time polymorphism, where the compiler determines which function call to bind at compilation. Runtime polymorphism is implemented using function overriding with **virtual functions**.

Function Overriding

Function Overriding occurs when a derived class defines one or more member functions of the base class. That base function is said to be **overridden**. The base class function must be declared as **virtual function** for runtime polymorphism to happen.

✚ Example of polymorphism at run time is **Method overriding**.

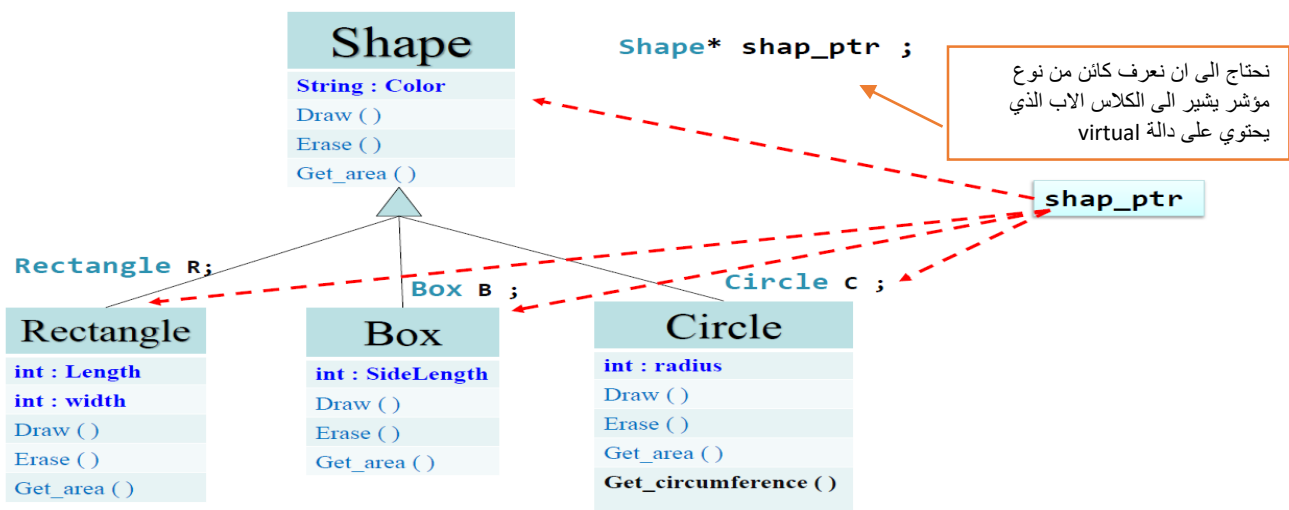
```
Box b;
b.Get_area();
```



6-3 Functions Overriding using Virtual Functions

A virtual function is a member function which is declared in a base class and is re-defined (Overridden) by a derived class. When you create a derived class object using a pointer to the base class, you can call a virtual function for that object and execute the derived class's version of the function.

• Virtual functions ensure that the correct function is called for an object, regardless of the type of reference (or pointer) used for function call.



Example:

```
#include <iostream>

using namespace std;

// Base Class: Shape
class Shape {
    string color ;
public:
    Shape(string c)
    {
        color=c;
    }
    virtual double area()
    {
        cout<<"parent class area: "<< endl;
    }
    virtual void draw()
    {
        cout<<"draw parent shape with color in: "<< color<<" color"<<endl;
    }
    virtual void eras()
    {
        cout<<"eras the parent shape : "<<endl;
    }
};

// Derived Class: Rectangle
class Rectangle : public Shape {
private:
    double length, width;
public:
    Rectangle(double l, double w, string c): Shape(c)
    {
        length =l;
        width = w;
    }
    double area() {
        cout <<"rectangle area: " << (width * length)<<endl ;
        return (width * length);
    }
    void draw()
    {
        cout << "Drawing a Rectangle" << endl;
    }
    void eras()
    {
        cout<<"the rectangle is erased : "<<endl;
    }
};
```

```

    }
};

// Derived Class: Circle
class Circle : public Shape {
private:
    double radius;
public:
    Circle(double r , string c) : Shape(c)
    {
        radius =r;
    }

    double area() {
        cout <<"circle area: " << 3.14 *(radius * radius) <<endl ;
        return (3.14 * radius * radius);
    }
    void draw()
    {
        cout << "Drawing a circle" << endl;
    }
    void eras()
    {
        cout<<"the Circle is erased : "<<endl;
    }
};

int main() {
    // Polymorphism: Using a base class pointer to hold derived class objects
    Shape* shap_ptr;
    Rectangle rec(10.0, 7.0, "red");
    Circle cir(5.0,"yellow");
    shap_ptr=&rec;
    shap_ptr->area();
    shap_ptr->draw();

    shap_ptr=&cir;
    shap_ptr->area();
    shap_ptr->draw();

    return 0;
}

```

```

rectangle area: 70
Drawing a Rectangle
circle area: 78.5

```

If we didn't use the virtual keyword (remove them from program) the output will be.

```

parent class area:
draw parent shape with color in: red color
parent class area:
draw parent shape with color in: yellow color

```